RAPPORT DE PROJET

Jeu Vidéo 3D d'Action-Aventure

— Spyro-Like —

Moteur de jeu : **Godot 4 | GDScript**
Langage : **GDScript**

Ce document constitue le rapport complet du projet de jeu vidéo de type action-aventure 3D, réalisé sous Godot 4 avec le langage GDScript. Il couvre la présentation générale du jeu, l'architecture technique, les fonctionnalités actuelles, ainsi que les évolutions planifiées.

Sommaire

1. Présentation générale du jeu
2. Concept et piliers narratifs
3. Gameplay et mécaniques de jeu
4. Architecture technique
5. Description des scripts
6. Système d'ennemis et boss
7. Système de collectibles
8. Structure des niveaux
9. Interface utilisateur (HUD)
10. Fonctionnalités actuelles — résumé
11. Fonctionnalités à venir
12. Conclusion

1. Présentation générale du jeu

Ce projet est un jeu vidéo d'action-aventure 3D développé avec Godot 4, directement inspiré du jeu Spyro the Dragon. Le joueur incarne un jeune dragon capable de courir, sauter, planer, cracher du feu et charger ses ennemis. L'objectif principal est de traverser plusieurs niveaux, collecter des gemmes, éliminer des ennemis variés et affronter un boss final multi-phases pour sauver son monde.

Le jeu repose sur une boucle de gameplay simple et efficace :

- Explorer les niveaux ouverts
- Collecter les gemmes pour débloquer de nouveaux portails
- Combattre des ennemis avec différentes stratégies
- Progresser vers le boss final

Pitch : Un jeune dragon doit restaurer l'énergie de son monde en collectant des gemmes et en libérant des gardiens transformés en statues par un tyran. Seul contre tous, il devra maîtriser le feu, la charge et le vol pour triompher.

Le projet est actuellement en phase de prototype avancé. Plusieurs niveaux jouables sont disponibles avec leurs ennemis, leurs gemmes et leurs portails de progression. Le boss final est fonctionnel et dispose d'un système de phases. L'ensemble du code est structuré de façon modulaire pour faciliter les extensions futures.

2. Concept et piliers narratifs

2.1 L'univers

L'ambiance visée est onirique, colorée et entraînante. Le monde est constitué de plusieurs zones thématiques reliées par un hub central (le Jardin). Chaque zone possède sa propre identité visuelle et ses propres défis. Le joueur y retrouve une atmosphère proche des jeux Spyro originaux, avec des environnements vastes, des ennemis hauts en couleur et une musique dynamique.

2.2 Le personnage principal

Le héros est un dragon jouable en vue troisième personne. Ses capacités définissent entièrement la façon d'interagir avec le monde :

- Marche et course fluides
- Charge avec impact sur les ennemis et les coffres
- Saut et plané contrôlé
- Attaque de feu (boules de feu à courte portée)
- Attaque mêlée (dash corps à corps)
- Attaque roulade
- Vol libre en l'air avec montée, descente et déplacement horizontal

2.3 La boucle narrative

La progression suit un schéma classique des jeux platformers 3D : le joueur part d'un hub world neutre, entre dans des niveaux via des portails, accumule des ressources et des victoires pour débloquer le chemin vers le boss final. Une fois le boss vaincu, le monde est restauré.

3. Gameplay et mécaniques de jeu

3.1 Contrôles

Voici la liste complète des contrôles du jeu, configurés via l'InputMap de Godot :

Action	Touche	Effet
Avancer / Reculer	Z / S	Déplacement avant/arrière
Latéral	Q / D	Déplacement gauche/droite
Saut / Plané	Espace	Saut au sol, activation du vol en l'air
Charge	Shift	Accélération avec dégâts de contact
Attaque Feu (boule)	Clic Gauche	Lance une boule de feu en direction visée
Attaque Mêlée	Clic Droit	Dash + frappe au corps à corps
Descente vol	Ctrl	Descendre pendant le vol / plané
Stopper le vol	Double espace	Arette le vol et laisse spyro tombé

3.2 Système de mouvement

Le déplacement du dragon est géré par la classe Player (CharacterBody3D). La vitesse de marche, de course et de charge sont exportées pour être réglables dans l'éditeur. Le système intègre :

- Une interpolation fluide des vitesses via lerp()
- La gravité appliquée en continu en dehors du sol
- Un système d'endurance (stamina) pour la charge
- Une gestion de l'état machine via l'enum State (GROUNDED, JUMPING, GLIDING, CHARGING, FLAMING, ATTACKING, LANDING)

3.3 Le vol et le plané

L'un des mécanismes les plus élaborés du jeu est le système de vol. Lorsque le dragon est en l'air, il peut activer le mode GLIDING en appuyant à nouveau sur Espace. En mode plané :

- Maintenir Espace fait monter le dragon
- Maintenir Ctrl le fait descendre
- Sans input vertical, le dragon reste à altitude constante
- Une animation de pitch (inclinaison) du modèle reflète la direction de vol

Note technique : Le plané utilise glide_vertical_accel pour accélérer progressivement, et move_toward() pour amortir la vitesse verticale. Les angles de pitch sont calculés en degrés et convertis via deg_to_rad() pour une animation propre.

3.4 Les attaques

Le dragon dispose de deux types d'attaques distinctes :

Attaque de feu (clic gauche)

Déclenche la méthode _shoot_fireball() qui instancie une scène PlayerFireball.tscn et lui applique une direction et une vitesse. Une Area3D (FlameArea) gère la détection de collision. Cette attaque est soumise à un cooldown (fireball_cooldown).

Attaque mêlée (clic droit)

Déclenche un dash dans la direction du mouvement via trigger_right_click_attack(). La vitesse de dash est appliquée via lerp() sur la vélocité, et les dégâts sont distribués via la FlameArea en mode ATTACKING. La durée de l'attaque est contrôlée par un timer interne.

3.5 Système de charge

La charge est une accélération brutale activée avec Shift. Elle consomme de l'endurance (STAMINA_DRAIN_CHARGE) à chaque frame et se régénère automatiquement quand inutilisée. Si l'endurance atteint zéro, la charge se coupe et un booléen charge_exhausted empêche une réactivation immédiate.

4. Architecture technique

4.1 Organisation des scripts

Le projet est structuré autour d'une hiérarchie de classes claires :

- Global.gd (Autoload) : stocke les gemmes, signaux gems_changed, level_completed, mark_level_completed()
- Player.gd : contrôle complet du personnage, gestion des états, animations, collisions
- Enemy.gd (classe de base) : déplacement, santé, barre de vie 3D, détection du joueur
- Boss.gd : héritage Enemy, 3 phases, projectiles, animations, camera shake
- GemCollectible.gd : collecte, animation de vol vers la caméra, mise à jour de la gemme HUD
- LevelPortal.gd : conditions d'ouverture, rafraîchissement visuel, transition de scène
- CameraPreview.gd : plugin éditeur, aperçu Camera3D/2D dans le viewport

4.2 Autoload Global

Le singleton Global.gd centralise l'état persistant du jeu. Il expose :

- total_gems : int — nombre total de gemmes collectées
- Signal gems_changed(new_amount) — déclenché à chaque collecte
- Signal level_completed(level_id) — déclenché à la fin d'un niveau
- has_completed_level(id) et has_completed_all(ids) — vérifie la progression
- add_gems(amount) — ajoute des gemmes et émet le signal
- respawn_player(player) — téléporte le joueur au point de spawn

4.3 Système de signaux

Le projet exploite intensivement les signaux Godot pour le découplage des systèmes. Les principaux flux de signaux sont :

- GemCollectible.body_entered → Global.add_gems() → gems_changed → GemHUD
- Enemy.died → Level3Combat._on_enemy_died() → vérification conditions débloquage
- Boss.health_changed / phase_changed / boss_defeated → BossLevelController
- Global.gems_changed / level_completed → LevelPortal._refresh_visual_state()

4.4 Le système Tween

Les Tween de Godot sont utilisés à plusieurs endroits clés pour les animations procédurales :

- Gemmes : animation de vol vers la caméra (position, scale, emission) via tween.set_parallel()
- Boss : camera shake avec oscillation sur h_offset et v_offset de la Camera3D
- Panel caméra (plugin) : animation de retour en position avec EASE_OUT / TRANS_CUBIC
- BossLevelController : apparition/disparition de l'UI boss, fondu de la barre de vie
- Level3Combat : message de déverrouillage du portail avec fondu entrant et sortant
- CombatHint (Player) : popups de combat avec tween de scale + alpha

Exemple concret : Pour la collecte d'une gemme, trois propriétés sont tweenées en parallèle : global_position (courbe TRANS_CUBIC), scale (rétrécissement), et emission_energy_multiplier (éclat lumineux). Le tout se joue en 0.65 secondes.

4.5 Machine à états (State Machine)

Le personnage joueur utilise une enum State pour contrôler son comportement :

- GROUNDED : au sol, peut marcher, courir, charger
- JUMPING : en l'air après saut
- GLIDING : vol actif
- CHARGING : charge au sol
- FLAMING : animation de crache de feu en cours
- ATTACKING : dash mêlée en cours
- LANDING : animation d'atterrissage depuis le vol

La transition entre états est gérée dans update_state_after_motion() et chaque état détermine les animations jouées dans update_animations().

5. Description détaillée des scripts

5.1 player.gd

Script principal du joueur. Hérite de CharacterBody3D. Gère intégralement les mouvements, la physique, les attaques, les animations, l'endurance et la santé. Utilise un SpringArm3D pour la caméra en troisième personne avec collision automatique contre les murs.

Points techniques notables :

- floor_snap_length = 0.0 quand velocity.y > 0 pour éviter l'annulation du saut sur escaliers
- safe_margin = 0.08 pour améliorer la détection de collision sur plateformes fines
- Animation loops forcées via _set_animation_loop() pour éviter les freezes
- Spawn dynamique des fireballs via get_tree().current_scene.add_child()
- Vide détecté via void_fall_y et respawn via Global.respawn_player()

5.2 enemy.gd (base)

Classe de base pour tous les ennemis. Gère le déplacement vers le joueur, la détection de portée, les attaques périodiques, les dégâts reçus et la mort. Inclut la création procédurale d'une barre de vie 3D (QuadMesh) qui se tourne vers la caméra à chaque frame.

La barre de vie change de couleur dynamiquement : verte à plein HP, puis orange/rouge selon la valeur (1 - ratio). Les ennemis héritant d'Enemy peuvent surcharger is_vulnerable_to_damage_type() pour définir leurs immunités.

5.3 boss.gd

Hérite de Enemy. Système de boss à trois phases avec des comportements distincts :

- Phase 1 (survie) : invulnérable, bombarde l'arène de projectiles, durée = phase1_survival_duration
- Phase 2 : vulnérable au feu uniquement
- Phase 3 : vulnérable à la charge uniquement

Les projectiles sont spawnés en deux vagues : une pluie aléatoire sur toute l'arène, puis une salve ciblée autour du joueur avec un rayon qui se resserre quand la santé du boss baisse.

Le camera shake est implémenté directement dans le Boss via _trigger_camera_shake(), en tweenant les propriétés h_offset et v_offset de la Camera3D active.

5.4 gem_collectible.gd

Gère la collecte des gemmes avec une animation de vol vers la caméra. Quatre types de gemmes (RED, GREEN, BLUE, YELLOW) avec des valeurs et couleurs distinctes. L'émission lumineuse de la gemme est animée au moment de la collecte pour un effet visuel satisfaisant.

5.5 level_portal.gd

Portail de transition entre les niveaux. Vérifie les conditions (gemmes requises, niveaux complétés) avant d'autoriser l'entrée. Affiche en Label3D le statut d'ouverture ou les prérequis manquants. Les labels se tournent vers la caméra à chaque frame (billboard manuel).

5.6 camera_preview (plugin éditeur)

Plugin éditeur (@tool) qui affiche un aperçu en temps réel de la Camera3D ou Camera2D sélectionnée dans le viewport Godot. Supporte le redimensionnement par drag, le verrouillage de position, et l'animation de retour en place. Compatible avec les échelles d'écran HiDPI.

5.7 animation_controller.gd

Script de contrôle de l'AnimationTree du joueur alternatif. Interpole les blend positions (WalkBlend, RunBlend, SprintBlend) en fonction de input_dir, et gère les transitions WalkRunBlend et SpeedBlend de façon progressive via lerp().

5.8 camera_free.gd

Caméra libre de l'éditeur / mode débogage. Permet la navigation libre dans la scène avec la souris et les touches directionnelles. La rotation est clampée sur l'axe X pour éviter le gimbal lock.

6. Système d'ennemis et boss

6.1 Tableau récapitulatif

Ennemi	PV	Vitesse	Dégâts	Vulnérabilité
EnemyFroussard	2	5.2	1	Feu — fuit si charge détectée (ne marche pas complètement pour le moment)
EnemySoldier	4	3.6	1	Charge uniquement (bouclier anti-feu)
EnemyColossus	8	2.8	2	Feu uniquement (trop grand pour charge)
EnemySpider	3	5.8	1	Feu et charge
Boss (Calmaramon)	100	5.4	2-8	Phase 1 : immuniser a tout -- Phase 2 : feu — Phase 3 : charge

6.2 Ennemis de base

EnemyFroussard

Ennemi rapide (5.2 unités/s) qui prend la fuite si le joueur est en état de charge dans un rayon de 12 unités. La méthode _get_move_direction() est surchargée pour inverser la direction de fuite. Dispose d'animations spider dynamiques (idle, walk, attack) avec résolution automatique des noms d'animations.

EnemySoldier

Porte un bouclier (is_shielded = true), ce qui le rend immunisé au feu. Seule la charge peut l'éliminer. Vitesse modérée (3.6), 4 PV.

EnemyColossus

Géant (is_giant = true) immunisé à la charge. Doit être brûlé. Slow mais résistant (8 PV, 2.8 unités/s). Scale corporel augmenté (1.35x).

EnemySpider

Ennemi standard avec modèle 3D de spider et résolution automatique des animations. Aggressif, vitesse élevée (5.8), vulnérable aux deux types d'attaques.

6.3 Le Boss — Calmaramon

Le boss final est un adversaire multi-phases implémenté dans boss.gd. Il utilise un modèle 3D importé (calmaramon.glb) avec résolution automatique des animations par keyword matching (idle, walk, attack, hurt, death).

Fonctionnement des trois phases :

- **Le boss est invulnérable. Le joueur doit survivre pendant 18 secondes face à une pluie de projectiles. Un timer est affiché à l'écran.** Phase 1 — Survie :
- **Vulnérable aux attaques de feu uniquement. La densité des projectiles augmente de 20% (phase2_density_multiplier).** Phase 2 — Feu :
- **Vulnérable à la charge uniquement. Les projectiles accélèrent encore (+45%). La salve ciblée autour du joueur se resserre.** Phase 3 — Charge :

Système de projectiles adaptatif : Le nombre de projectiles par vague est calculé selon (1 - health_ratio), ce qui signifie que plus le boss a peu de vie, plus il bombarde intensément. La salve ciblée réduit aussi son rayon minimum quand les HP baissent.

7. Système de collectibles — Les Gemmes

7.1 Types de gemmes

Couleur	Valeur	Description

Rouge	1 pt	Gemme de base, la plus commune
Verte	2 pts	Gemme intermédiaire
Bleue	5 pts	Gemme rare, difficile d'accès
Jaune	10 pts	Gemme très rare, souvent cachée

7.2 Mécanique de collecte

Chaque gemme tourne sur elle-même et oscille verticalement (animation procédurale dans _process() via sin()). Quand le joueur entre en contact (body_entered), la gemme :

- Se désactive immédiatement (monitoring = false, collision disabled)
- Appelle Global.add_gems() pour mettre à jour le compteur
- Lance une animation Tween vers la caméra (position + scale + émission)
- Se supprime une fois l'animation terminée (queue_free())

7.3 Affichage HUD

Le GemHUD (Label) est connecté au signal gems_changed de Global. À chaque collecte, il met à jour son texte avec le format '💎 X'. Léger et découplé — il n'a besoin que d'une connexion de signal.

8. Structure des niveaux

8.1 Hub World — Le Jardin

Zone sans ennemis qui sert de point central. Contient les portails vers les trois niveaux principaux et le boss. Les portails sont verrouillés par défaut et s'ouvrent selon les conditions définies dans LevelPortal.

8.2 Niveau 1 — Tutoriel

Introduction aux mécaniques de base. Le joueur apprend à se déplacer, sauter et charger des coffres. Aucune condition de gemmes requise (portail ouvert depuis le début). Ennemis simples en faible nombre.

8.3 Niveau 2 — Plateformes

Apprentissage du plané au-dessus du vide. Platefomes espacées qui nécessitent de maîtriser le vol. Condition d'accès : 50 gemmes accumulées.

8.4 Niveau 3 — Combat

Niveau centré sur le combat. Contient 3 ennemis à éliminer pour ouvrir le portail vers le boss. Géré par Level3Combat.gd qui suit les ennemis via le signal died et déverrouille le portail une fois tous éliminés. Condition d'accès : 150 gemmes.

8.5 Niveau Boss

Arène fermée avec le boss Calmaramon. Condition : avoir complété les 3 niveaux précédents (required_levels). Une UI dédiée (BossLevelController) affiche la barre de vie du boss, son nom, la phase actuelle et le timer de survie en Phase 1.

8.6 Portails de niveau

Les portails LevelPortal sont des Area3D qui détectent l'entrée du joueur. Chaque portail expose :

- target_scene : chemin vers la prochaine scène .tscn
- gems_required : nombre minimum de gemmes
- required_levels : liste d'identifiants de niveaux complétés
- complete_level_on_enter : marque un niveau comme terminé à l'entrée

---

9. Interface Utilisateur (HUD)

9.1 Éléments actuels

- Compteur de gemmes : icône + nombre en temps réel via signal
- Barre d'endurance : se remplit/vide selon la charge, visible seulement quand nécessaire
- Barre de vie joueur : HP sur 5 points, visible en permanence
- UI Boss : barre de vie, nom, phase, timer Phase 1 — construite procéduralement dans BossLevelController
- Popups de combat : messages contextuels (ex : 'Ennemi immunisé au feu') avec animation Tween
- Message de déverrouillage : texte animé quand le portail boss s'ouvre

9.2 Construction procédurale de l'UI

Une grande partie de l'UI est créée par code plutôt que dans l'éditeur, notamment l'UI du boss et les hints de combat. Cette approche utilise CanvasLayer, Panel, Label, ProgressBar instanciés et configurés dans _ensure_boss_ui() et _ensure_combat_hint_ui(). Les couleurs, tailles et positions sont définies par code pour un contrôle total.

---

10. Fonctionnalités actuelles — Résumé

Statut : Prototype jouable avec tous les systèmes de base fonctionnels. Le jeu est traversable du début à la fin.

Gameplay

- Déplacement 8 directions avec marche, course, charge
- Saut, plané avec contrôle de l'altitude
- Attaque feu (boule de feu) avec cooldown
- Attaque mêlée (dash + dégâts)
- Système d'endurance pour la charge
- Invulnérabilité temporaire après dégâts reçus
- Respawn automatique en cas de chute dans le vide

Ennemis

- 4 types d'ennemis (Froussard, Soldier, Colossus, Spider)
- Comportements distincts (fuite, bouclier, résistance)
- Barre de vie 3D dynamique billboard
- Animations 3D avec résolution automatique

Boss

- Boss Calmaramon avec 3 phases
- Système de projectiles adaptatif
- Salve ciblée autour du joueur
- Camera shake procédural
- UI dédiée avec barre de vie et timer

Progression

- Système de gemmes (4 types, 4 valeurs)
- Portails conditionnels (gemmes + niveaux)
- 4 niveaux jouables + hub
- Persistance de la progression via Autoload Global

Technique

- Plugin éditeur de prévisualisation caméra
- Caméra libre pour le débogage
- AnimationTree avec blend spaces
- Système de signaux découplé

---

11. Fonctionnalités à venir

11.1 Potions dans l'arène du boss

L'une des prochaines évolutions majeures sera l'ajout de potions récupérables dans l'arène du boss final. Ces consommables apparaîtront à des positions fixes ou aléatoires dans l'arène et permettront de faciliter la victoire contre Calmaramon :

- Potion de soin : restaure des points de vie au joueur
- Potion d'endurance : recharge instantanément la barre de stamina pour permettre une charge critique
- Potion de bouclier temporaire : réduit les dégâts reçus pendant 10 secondes
- Potion de vitesse : augmente temporairement la vitesse de déplacement

Ces potions seront espacées dans l'arène pour forcer le joueur à se déplacer et prendre des risques pour les récupérer. Leur apparition sera liée à la phase du boss : plus le boss est en Phase avancée, plus des potions de soin apparaissent pour équilibrer la difficulté.

11.2 Amélioration des maps

Les niveaux actuels sont fonctionnels mais visuellement basiques. Une refonte graphique est prévue pour chaque zone :

- Niveau 1 — Tutoriel : ajout de décorations, objets interactifs, meilleure lisibilité des chemins
- Niveau 3 — Combat : arène plus complexe avec obstacles tactiques, zones surélevées et couloirs
- Arène du Boss : décor plus immersif avec piliers destructibles, effets de particules, éclairages dynamiques

L'objectif est de passer de niveaux de prototype à des maps finies avec une identité visuelle forte, tout en conservant les hitboxes et logiques de jeu existantes.

11.3 Techniques débloquables

Un système de progression des capacités est prévu. Chaque fois qu'un niveau est complété, le joueur débloquera une nouvelle technique ou amélioration :

- Fin Niveau 1 : Charge améliorée — la charge inflige des dégâts de zone lors de l'impact
- Fin Niveau 2 : Vol prolongé — l'endurance de vol est doublée
- Fin Niveau 3 : Double saut — capacité de sauter une deuxième fois en l'air
- Après le boss : Super Charge — capacité spéciale dévastatrice avec longue recharge

Ces déblocages seront gérés dans l'Autoload Global via des flags de progression, et les capacités seront vérifiées dans Player.gd avant leur exécution.

11.4 Augmentation du nombre de niveaux

La structure actuelle de 4 niveaux sera étendue à au moins 6-8 zones jouables :

- Niveau 4 (Glace) : environnement glacial, ennemis résistants au feu, nécessite la charge exclusive
- Niveau 5 (Volcanique) : plateformes mobiles sur lave, ennemis en armure ignifugée
- Niveau 6 (Céleste) : dans les nuages, level design orienté vol et plané
- Niveau de défi : niveau bonus sans checkpoint pour les joueurs expérimentés

Chaque nouveau niveau introduira au moins un nouvel ennemi ou une variante pour maintenir la variété du bestiaire.

11.5 Autres améliorations prévues

- Système audio : musiques de fond dynamiques, effets sonores pour collecte, attaques, dégâts
- Effets de particules : traînées de feu, étincelles sur charge, éclats de gemmes
- Menu principal et écran de chargement
- Sauvegarde persistante via des fichiers (actuellement en mémoire uniquement)
- Options de jeu : sensibilité souris, volume, résolution
- Minimap ou indicateurs directionnels des objectifs
- Ennemis patrouilleurs avec navigation NavMesh
- Coffres destructibles contenant des gemmes
- Ajoue des designs pour les enemies du tuto car pour le moment on a que une araigné

12. Conclusion

Ce projet de jeu vidéo 3D d'action-aventure constitue une réalisation technique solide, développée entièrement sous Godot 4 avec GDScript. En partant d'un cahier des charges inspiré de Spyro the Dragon entièrement conçu par moi-même , un prototype jouable a été construit avec des systèmes fonctionnels de déplacement, de combat, de progression et d'interface utilisateur.

Les technologies utilisées — machine à états, tweens procéduraux, signaux Godot, héritage de classes, plugins éditeur — illustrent une bonne maîtrise du moteur et des pratiques de développement de jeu vidéo. Le code est structuré pour être extensible : ajouter de nouveaux ennemis, niveaux ou mécaniques ne nécessite que d'étendre les classes existantes.

Les prochaines étapes visent à enrichir le contenu (niveaux, ennemis, potions, techniques), améliorer le rendu visuel et finaliser le polish (audio, effets de particules, menus). L'objectif final est un jeu complet, jouable du début à la fin, avec une progression satisfaisante et une courbe de difficulté équilibrée.

> **Vision long terme :** Transformer ce prototype en un jeu court mais complet (45-60 minutes de jeu), avec 6 niveaux, un boss final épique, des techniques débloquables et une identité visuelle cohérente — un vrai hommage à l'ère des platformers 3D des années 2000.

— Fin du rapport —